

Adversarial Attacks and Defenses on Graph-aware Large Language Models (LLMs)

Anonymous submission

Abstract

Large Language Models (LLMs) are increasingly coupled with graph-structured data to perform tasks such as node classification, a domain long dominated by Graph Neural Networks (GNNs). Although this integration exploits rich relational information to improve task performance, the adversarial robustness of the resulting graph-aware LLMs has remained unexamined. We present the first systematic study of these vulnerabilities, adapting established graph adversarial attack methods—spanning both poisoning (training-time) and evasion (test-time) regimes—to two representative and computationally efficient models built on frozen pre-trained LLMs, LLAGA (Chen et al. 2024) and GRAPH PROMPTER (Liu et al. 2024). Beyond transferring existing attacks, we uncover a previously unrecognized attack surface unique to LLAGA, in which an adversary injects malicious nodes into the placeholder slots of the node sequence template to severely degrade classification accuracy. Our analysis demonstrates that specific design choices in how graphs are encoded for LLMs directly govern adversarial susceptibility, establishing three findings: (1) the node sequence template in LLAGA substantially amplifies its vulnerability; (2) the GNN encoder in GRAPH PROMPTER confers markedly greater robustness; and (3) both paradigms are acutely and consistently susceptible to imperceptible feature perturbation attacks, a threat largely overlooked in the GNN literature. Building on these insights, we introduce GALGUARD, an end-to-end defense framework that unifies an LLM-based feature correction module, which counters feature-level perturbations, with adapted GNN defenses that protect against structural attacks¹

1 Introduction

Graphs, in which nodes and edges encode entities and the relationships among them, provide a natural abstraction for the complex structures that arise across a wide range of real-world domains, including social networks (Fan et al. 2019), biology (Dong, Mücke, and Khosla 2022), finance (Wang et al. 2021), and healthcare (Ahmedt-Aristizabal et al. 2021). Graph Neural Networks (GNNs) (Hamilton, Ying, and Leskovec 2017; Kipf and Welling 2022; Veličković et al. 2018), purpose-built for graph-structured data, have long set the standard on tasks such as node classification and link prediction. More recently, the striking success of large language

models (LLMs) such as GPT-4 (Achiam et al. 2023), Gemini (Team et al. 2023), LLaMA (Touvron et al. 2023; Dubey et al. 2024), Vicuna (Chiang et al. 2023), and PaLM-2 (Anil et al. 2023)—trained on vast textual corpora—has motivated a rapidly growing effort to harness their capabilities for graph-related tasks. This fusion of graph data with LLMs, termed *graph-aware LLMs* (Xie et al. 2023), has opened a distinct and active line of research in which LLMs are increasingly deployed on tasks traditionally handled by GNNs (Ye et al. 2023; Fatemi, Halcrow, and Perozzi 2023; Kuang et al. 2023; Zhang et al. 2023; Chen et al. 2023b, 2024). Indeed, graph-aware LLMs have been shown to *outperform* GNNs on a variety of graph-related tasks (Wenkel, Wolf, and Knyazev 2023; Chen et al. 2023b; Zhu et al. 2024; Chen et al. 2024), establishing them as compelling alternatives to the established GNN paradigm.

Yet despite this rapid adoption, the adversarial vulnerabilities of graph-aware LLMs remain almost entirely unexplored—a critical gap given that these models are increasingly considered for the very high-stakes settings in which GNNs already operate. Existing adversarial attacks on GNNs, encompassing both poisoning (training-time) and evasion (test-time) strategies (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Dai et al. 2023), operate in a black-box setting in which the attacker has no knowledge of the model’s architecture or parameters and influences it only through the graph data. Such attacks compromise *model integrity*, deliberately degrading predictive performance through subtle modifications to graph structure or node features. Whereas GNN research has concentrated overwhelmingly on structural attacks, the fundamentally textual nature of graph-aware LLMs elevates imperceptible perturbations of node features from a peripheral concern to a first-order threat—one that the prior literature has largely overlooked.

These considerations—the ongoing migration from GNNs to LLMs for graph tasks, the black-box character of current graph adversarial attacks, and the newly salient threat of feature perturbations—converge on the central question of this work: *do adversarial attacks designed for GNNs remain effective against the emerging graph-aware LLM paradigm, and what new vulnerabilities does that paradigm introduce?* Answering this question requires attention to how graphs are integrated into the LLM, because the effectiveness of graph-

¹Our code will be released upon acceptance.

aware LLMs *depends critically* on this integration. Whereas node features are readily consumed given the LLM’s proficiency with text, encoding the graph structure is substantially more involved and demands dedicated mechanisms to capture relational information (Chen et al. 2024; Guo et al. 2023; Liu and Wu 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024). We therefore organize existing graph-aware methods into two classes according to how they encode graphs for LLMs: one that renders graph data as *textual descriptions*, converting relational information into natural language, and another that employs *learned projectors* to bridge the inherent gap between the graph and text domains. Because conventional text-based adversarial attacks on LLMs already suffice to compromise models that consume purely textual graph descriptions, we direct our investigation at the more challenging and more consequential learned-projector class, instantiated by two representative and computationally efficient models, LLAGA (Chen et al. 2024) and GRAPH-PROMPTER (Liu et al. 2024), both of which operate on a frozen pre-trained LLM.

Our analysis demonstrates that specific design choices in how graphs are encoded for LLMs directly determine—and can markedly amplify—adversarial susceptibility. In particular, we identify a previously unrecognized attack surface in LLAGA that enables an attacker to severely degrade performance by injecting malicious nodes into the placeholder slots of the node sequence template. Building on this analysis, we propose an end-to-end graph-aware LLM defense, GALGUARD, which couples an LLM-based feature corrector with adapted GNN-based structural defenses to deliver robust, dual-pronged protection. To our knowledge, this is the first study to systematically characterize the adversarial vulnerability of graph-aware LLMs and to propose a defense tailored to this paradigm. We summarize our contributions as follows:

- **A taxonomy of graph-aware LLMs.** We organize existing graph-aware LLMs by their graph-encoding strategy, clarifying how different methodologies integrate graph structure into LLMs and pinpointing where each is exposed to adversarial manipulation.
- **Transferability of GNN attacks.** We adapt two representative GNN adversarial attacks, NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019), and establish that poisoning and, especially, evasion attacks transfer to graph-aware LLMs, with susceptibility governed by the encoding design.
- **A new attack surface in LLAGA.** We discover that the node sequence template exposes an exploitable attack surface: injecting malicious nodes into its placeholder slots degrades performance more severely than conventional structural attacks.
- **Feature-level vulnerability.** We provide the first holistic assessment of imperceptible attacks on textual node features—a perspective overlooked in GNN research—and show that both models are acutely vulnerable to them.
- **The GALGUARD defense.** We propose GALGUARD, a novel end-to-end defense that integrates LLM-based feature correction with adapted GNN-based structural de-

fenses, consistently restoring robustness across datasets and attack types.

2 Graph Encoding-Based Taxonomy of LLM Adaptations

Graph-encoding methods for LLMs fall into two principal classes: *textual descriptions of graphs*, in which relational information is expressed as natural language, and *learned graph projectors*, in which a trained projector maps graph data into embeddings or structured forms that the LLM can consume directly. We characterize each class in turn; Figure 5 illustrates both. A full account appears in Appendix A.1, and related work on GNN attacks in Appendix A.2.

Graph Representation as a Textual Description. This class converts graph data into natural language so that LLMs can process it as text, typically paired with task-specific querying instructions (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a; Zhang 2023). In multimodal settings, Liu et al. (2023) and Su et al. (2022) aligned molecular graphs with textual descriptions. Complementarily, Huang et al. (2023a) examined the incorporation of graph structure into LLM prompts and found that LLMs tend to treat graph-related prompts as contextual paragraphs rather than as explicit relational structures—a result that underscores the decisive role of prompt design, as further explored in (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024; He et al. 2024). A parallel line of work uses LLMs to enrich node features and graph structures, to act as direct predictors, or to serve as feature extractors for GNNs (Chen et al. 2023a; Chandra et al. 2020; Chen et al. 2023b; Mavromatis et al. 2023). For inference, Zhu et al. (2024) proposed a fine-tuning method for textual graphs that combines LLMs and GNNs, and Duan et al. (2023) introduced an effective approach for enhancing textual graph learning. Although these methods offer a direct route to applying LLMs on graph tasks, they struggle to capture fine-grained structural detail—particularly for large graphs—and remain heavily contingent on prompt engineering.

Graph Representation as a Learned Projector. To overcome the limitations of textual representations, learned-projector methods transform graph data into embeddings or graph vectors that the LLM can consume more directly. Representative instances include LLAGA, which couples node-level templates with a linear projector (Chen et al. 2024), and methods that inject GNN layers into a pre-trained LLM (Liu et al. 2024; Qin et al. 2023). Related approaches encode knowledge graphs via graph neural prompts (Tian et al. 2024), build prompt-based node feature extractors with a GNN adapter (Huang et al. 2023b), or condense graph information into fixed-length prefixes using graph transformers (Chai et al. 2023; Yang et al. 2021; Peng et al. 2024).

This Work: We concentrate on projector methods that operate on *frozen pre-trained LLMs*, since they are computationally efficient and avoid the substantial cost of retraining (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). As representative and complementary

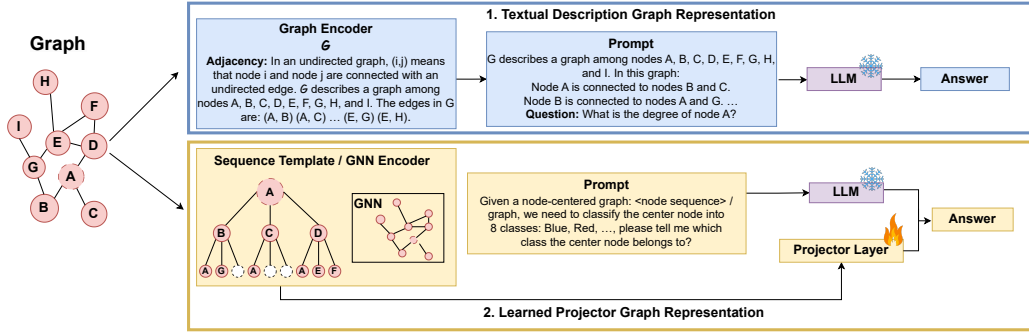


Figure 1: **Graph encoding-based adaptations of LLMs.** Under the textual-description representation, graphs are rendered as natural language (top). Under the learned-projector representation, graphs are encoded either by node templates that serialize the graph into sequences or by GNNs (bottom). In both cases the LLM remains frozen; only the projector that maps the graph into tokens is trained.

instances of this class, we study LLAGA (Chen et al. 2024), which pairs node-level templates with a linear projector, and GRAPHPROMPTER (Liu et al. 2024), which pairs a GNN encoder with a linear projector. We exclude textual-description methods because they scale poorly and, crucially, introduce no novel graph-specific attack surface: expressed as plain text, they are exposed only to conventional text-based adversarial attacks rather than to the structural and feature perturbations we set out to characterize.

3 Threat Model

We ground our threat model in the well-established literature on graph adversarial attacks and extend it to the graph-aware LLM setting (Zügner, Akbarnejad, and Günnemann 2018; Wu et al. 2019; Zügner and Günnemann 2019; Ma, Ding, and Mei 2020; Jin et al. 2021; Dai et al. 2023; Li et al. 2023). The model reflects realistic adversarial constraints. In social networks, for instance, an attacker typically controls only a handful of bot accounts; this limited access maps directly onto our setting, in which the adversary manipulates a small subset of nodes to evade detection or to influence the classification of legitimate users. Wikipedia offers a second instance: hoax articles with sparse links to legitimate content can be strategically connected to genuine articles so as to corrupt their category assignments. The concern is heightened by the fact that graph-aware LLMs are increasingly candidates for deployment in critical applications—recommendation systems, social media analysis, and even healthcare—where they may supplant GNNs. Following established practice, we specify the threat model along three dimensions—model access, adversarial capabilities, and attack strategies—and then state the attacker’s goal.

Model Access. The attacker operates in a strict black-box setting, with no knowledge of the model’s architecture or parameters, and can influence the model only indirectly by acting on the training or test data.

Adversarial Capabilities. Consistent with standard poisoning (respectively, evasion) attacks, the attacker has full access to the target model’s training (respectively, inference) data. Subject to a predefined perturbation budget and to unnotice-

ability constraints that preserve evasiveness, the attacker may modify or inject data.

Attack Strategies. The attacker employs two complementary strategies: feature manipulation, which alters node features, and structure manipulation, which adds or removes edges.

Attacker’s Goal. The attacker seeks to compromise model integrity by degrading overall predictive performance—concretely, by increasing the misclassification rate of the node classifier.

4 Experimental Setup

Dataset. We evaluate on three widely used text-attributed graph benchmarks—Cora, Citeseer, and PubMed (Yang, Cohen, and Salakhudinov 2016)—which span small-to-medium scales and a range of sparsities and serve as standard testbeds for the models we study (Chen et al. 2023a, 2024; Liu et al. 2024). To further establish that our findings hold at scale, we additionally evaluate on the large-scale ArXiv dataset (Hu et al. 2020) (see Appendix A.5). Following (Chen et al. 2024), we adopt standard train/validation/test splits of 6:2:2 for the three smaller datasets and 5:2:3 for ArXiv. Detailed statistics are reported in Table 1.

Table 1: Dataset statistics.

Data	#Node	#Edge	Sparsity (%)
Cora	2708	5429	14.81
CiteSeer	3327	4614	8.34
PubMed	19717	44338	2.28

Tasks. Our primary task is *node classification*, the canonical benchmark for graph machine learning models and the standard setting in which graph adversarial attacks are evaluated (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Liu and Wu 2023).

Evaluation Metrics. We report *accuracy*, the proportion of correctly predicted node labels. Each experiment is repeated three times, and we report the mean and standard deviation.

All experiments were conducted on a machine equipped with an NVIDIA A40 GPU with 48 GB of memory.

Choice of Graph-aware LLM and Adversarial Attack Methods. We now justify our selection of graph-aware LLMs and adversarial attack methods for probing the vulnerabilities of the graph-aware LLM paradigm.

Graph-Aware LLM Methods. We study two representative and complementary graph-aware LLMs, LLAGA (Chen et al. 2024) and GRAPHPROMPTER (Liu et al. 2024). Both operate on a *frozen pre-trained LLM*, rendering them computationally efficient and readily adaptable across graph tasks and LLM backbones. LLAGA encodes structure through node-level templates coupled with a linear projector, whereas GRAPHPROMPTER encodes it through a GNN-based encoder coupled with a linear projector; in both cases the projector is a two-layer multi-layer perceptron (MLP). The foundation LLMs for LLAGA and GRAPHPROMPTER are Vicuna-7B (Chiang et al. 2023) and LLaMA2-7B (Touvron et al. 2023), respectively. This pairing lets us isolate how the graph-encoding mechanism—template-based versus GNN-based—shapes adversarial robustness while holding the frozen-LLM design fixed.

Adversarial Attack Methods. We conduct our investigation with two representative graph adversarial attacks, NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019), exercising each under both poisoning and evasion regimes. NETTACK is a targeted attack that computes minimal perturbations to misclassify a specific node, whereas METAATTACK is an untargeted attack that treats the graph structure as a hyperparameter to be optimized. Following (Li et al. 2023; Zügner and Günnemann 2019), we convert NETTACK into an untargeted evaluation by sequentially targeting each test node and aggregating the outcomes. Across all experiments we perturb only 10% of the edges, thereby enforcing the unnoticeability constraint of (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019). Full details are provided in Appendix A.2.

5 Results

5.1 Poisoning and Evasion Attacks on Graph-aware LLMs

We analyze poisoning and evasion attacks by adapting NETTACK (Zügner and Günnemann 2019) and METAATTACK (Zügner and Günnemann 2019) to each model. For poisoning LLAGA, we generate the node-level sequences from the perturbed graph and then retrain the projector; for poisoning GRAPHPROMPTER, we re-encode the perturbed graph with a GAT and retrain its projector. For evasion, both models are trained on unperturbed data and the perturbations are introduced only at inference.

Results. The results in Table 2 establish three consistent findings. First, poisoning attacks have only a limited effect on graph-aware LLMs, degrading accuracy by at most 11% under either attack; this resilience follows from the robustness of the learned representations together with the additional context the frozen LLM contributes. Evasion attacks, by contrast, are decisively more effective, inducing degradations of up to

51%—a striking asymmetry that identifies inference-time perturbation as the dominant structural threat to this paradigm. Second, across every dataset and both regimes, METAATTACK consistently surpasses NETTACK, a direct consequence of its treating graph edges as optimizable hyperparameters and thereby crafting small yet highly effective perturbations. Third, attack effectiveness scales with graph density: denser graphs are markedly more vulnerable than sparse ones, as illustrated by LLAGA’s 51% accuracy drop on the denser Cora under evasion versus only 19% on the sparser PubMed. Together these findings show that transferred GNN attacks not only remain effective against graph-aware LLMs but degrade them in structured, predictable ways.

5.2 Which Graph-aware Paradigm is More Susceptible to Graph Adversarial Attacks and Why?

Our experiments (Table 2) establish that LLAGA is substantially more susceptible to structural attacks than GRAPHPROMPTER, and they attribute this gap directly to LLAGA’s *heightened sensitivity to structural perturbations*. LLAGA encodes graph structure in two coupled ways: it generates a fixed-length node-level sequence for each node and computes a graph Laplacian embedding, with every position in the sequence corresponding uniquely to a relative structural position in the original graph. Because these node embeddings are derived directly from the graph structure, structural perturbations propagate immediately into the representation the LLM consumes, and accuracy degrades accordingly. GRAPHPROMPTER, in contrast, leaves its textual node embeddings intact under structural perturbation—only the GNN component is affected—so the unperturbed textual features augment the perturbed structure and materially buffer the attack. This contrast identifies the encoding mechanism itself, rather than the choice of attack, as the decisive determinant of structural robustness.

6 Node Sequence Template Injection Attacks

We now expose a new attack surface in LLAGA that arises from the neighborhood detail template used to construct its node sequences. The attack is illustrated in Figure 2, and the corresponding algorithm is given in Appendix A.4. To obtain the node sequence for a node u , LLAGA’s neighborhood detail template first builds a *fixed-shape* computational tree in which the children of each node are its sampled 1-hop neighbors, recursively sampling neighbors at every subsequent level. When a node has fewer neighbors than required, placeholder tokens fill the remaining slots to preserve the fixed child count (e.g., $k = 10$), in accordance with LLAGA’s original design (see “Clean” in Figure 2 and lines 1–14 of the algorithm). Precisely this fixed-shape requirement—and the placeholders it necessitates—constitutes the exploitable attack surface.

By manipulating the graph so that adversarially chosen nodes occupy these placeholder slots, an attacker can substantially degrade model performance. We instantiate this idea with three progressively more aggressive strategies. *Non-Adjacent Injection (NI)* injects nodes lying outside the 2-hop

Table 2: Node classification accuracy of the graph-aware LLMs LLAGA and GRAPH PROMPTER under NETTACK and METAATTACK on Cora, Citeseer, and PubMed, for both poisoning and evasion attacks. Parenthetical values report the percentage drop from clean accuracy; higher drops indicate more effective attacks. Evasion attacks are consistently far more damaging than poisoning, and LLAGA degrades more sharply than GRAPH PROMPTER.

Attack	Type	Cora		Citeseer		PubMed	
		LLAGA	GRAPH PROMPTER	LLAGA	GRAPH PROMPTER	LLAGA	GRAPH PROMPTER
Clean	—	0.89 $\pm$ 0.07	0.60 $\pm$ 0.03	0.71 $\pm$ 0.04	0.70 $\pm$ 0.03	0.90 $\pm$ 0.04	0.90 $\pm$ 0.02
NETTACK	Poisoning	0.87 $\pm$ 0.04 (2%)	0.60 $\pm$ 0.03 (0%)	0.64 $\pm$ 0.05 (10%)	0.68 $\pm$ 0.02 (3%)	0.89 $\pm$ 0.03 (1%)	0.88 $\pm$ 0.02 (2%)
	Evasion	0.55 $\pm$ 0.09 (38%)	0.53 $\pm$ 0.05 (12%)	0.59 $\pm$ 0.04 (19%)	0.61 $\pm$ 0.03 (14%)	0.84 $\pm$ 0.05 (7%)	0.81 $\pm$ 0.02 (10%)
METAATTACK	Poisoning	0.79 $\pm$ 0.03 (11%)	0.58 $\pm$ 0.05 (3%)	0.63 $\pm$ 0.04 (12%)	0.65 $\pm$ 0.06 (7%)	0.89 $\pm$ 0.08 (1%)	0.85 $\pm$ 0.02 (6%)
	Evasion	0.44 $\pm$ 0.06 (51%)	0.52 $\pm$ 0.04 (13%)	0.50 $\pm$ 0.03 (35%)	0.56 $\pm$ 0.02 (22%)	0.73 $\pm$ 0.04 (19%)	0.77 $\pm$ 0.03 (14%)

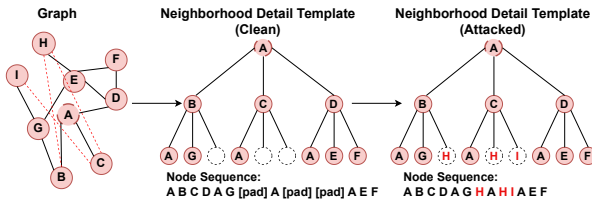


Figure 2: Illustration of the node sequence template injection attacks, shown here for the Non-Adjacent Injection (NI) attack. Red dashed lines denote the edges added by the attacker. For the Supernode Injection (SI) and Multiple Supernode Injection (MSI) attacks, one or more high-degree supernodes are injected into the placeholder slots instead. Here, $k = 3$.

neighborhood of the target node u (see "Attacked" in Figure 2 and lines 21–27). *Supernode Injection (SI)* instead injects a single high-degree node into a placeholder slot, again constrained to lie outside the target’s 2-hop neighborhood. *Multiple Supernode Injection (MSI)* generalizes SI by distributing multiple distinct supernodes across the available placeholders (lines 35–43). The mechanism common to all three is that uninformative nodes inserted into the node sequence corrupt the target node’s representation, which in turn yields a poorly trained projection layer and compromises the integrity of both learning and inference.

Results. As reported in Table 3, LLAGA remains highly resilient to the injection attacks under poisoning on Cora and Citeseer, yet collapses under evasion: accuracy falls to 0.42, 0.36, and 0.30 on Cora and to 0.36, 0.30, and 0.24 on Citeseer for the NI, SI, and MSI attacks, respectively. The monotonic ordering NI > SI > MSI in residual accuracy confirms that injecting higher-degree and more numerous supernodes produces progressively greater corruption of the node sequence.

On the larger PubMed graph the same attacks are far more contained, reducing evasion accuracy only modestly (to 0.85, 0.82, and 0.76). This gap has a clear structural explanation: in the smaller, tightly connected Cora and Citeseer graphs a localized injection exerts a concentrated effect that severely disrupts the model’s local-neighborhood reasoning, whereas on the much larger PubMed graph the same perturbation is

Table 3: Accuracy of LLAGA under the node sequence template injection attacks across datasets and regimes. NI, SI, and MSI denote the non-adjacent, single-supernode, and multiple-supernode injection attacks, respectively; Clean reports unattacked performance. Evasion is far more damaging than poisoning, and the effect intensifies from NI to MSI.

Data	Attack	NI	SI	MSI
Cora	Clean	—	0.89 $\pm$ 0.07	—
	Poisoning	0.86 $\pm$ 0.03	0.84 $\pm$ 0.06	0.82 $\pm$ 0.04
	Evasion	0.42 $\pm$ 0.08	0.36 $\pm$ 0.05	0.30 $\pm$ 0.05
Citeseer	Clean	—	0.71 $\pm$ 0.04	—
	Poisoning	0.69 $\pm$ 0.05	0.66 $\pm$ 0.03	0.62 $\pm$ 0.02
	Evasion	0.36 $\pm$ 0.06	0.30 $\pm$ 0.04	0.24 $\pm$ 0.05
PubMed	Clean	—	0.90 $\pm$ 0.04	—
	Poisoning	0.88 $\pm$ 0.04	0.85 $\pm$ 0.05	0.81 $\pm$ 0.03
	Evasion	0.85 $\pm$ 0.07	0.82 $\pm$ 0.04	0.76 $\pm$ 0.03

diluted across a far richer neighborhood, blunting its impact.

Critically, the injection attacks are more effective than conventional graph adversarial methods. Under evasion, MSI drives accuracy down to 0.30, 0.24, and 0.76 on Cora, Citeseer, and PubMed, whereas METAATTACK reaches only 0.44, 0.50, and 0.73 (Table 2); on the two denser graphs the template-specific attack is thus substantially more damaging than the strongest transferred structural attack, underscoring that the node sequence template introduces a genuinely new and more potent vulnerability.

7 Poisoning and Evasion Attack via Feature Perturbation

Existing graph adversarial attacks concentrate almost exclusively on structural perturbations (Li et al. 2023; Wu et al. 2019; Dai et al. 2018; Sun et al. 2022; Zügner and Günnemann 2019); the one feature-oriented exception, NETTACK (Zügner, Akbarnejad, and Günnemann 2018), does not transfer to the textual domain. Because structural attacks achieve only moderate success against robust models such as GRAPH PROMPTER, and because our analysis has already shown textual features to be pivotal for robustness, we now target those features directly through imperceptible feature

perturbation attacks.

We build on the imperceptible attack strategies of (Boucher et al. 2022), but introduce a key modification: we drive the perturbations with a task-agnostic translation objective (English to French). The objective seeks input edits that are visually indistinguishable to a human yet induce an incorrect translation, thereby corrupting the encoded semantics—the premise being that a corrupted translation signals correspondingly corrupted internal semantics. We instantiate two imperceptible attacks: *Homoglyphs*, which substitutes characters with visually near-identical Unicode counterparts, and *Reorderings*, which insert directional control characters to alter the rendered character order. Examples of both appear in Appendix A.6.

Following (Boucher et al. 2022), we optimize these perturbations with differential evolution (Storn and Price 1997), maximizing the Levenshtein distance between the model’s outputs on the perturbed and unperturbed text. The perturbation budget is fixed at 10% of the average length of each textual feature, preserving imperceptibility.

Results. Table 4 shows that Reordering attacks are uniformly more potent than Homoglyph attacks across every model and dataset. On Cora and CiteSeer, LLAGA is vulnerable to both: under poisoning, Homoglyph reduces accuracy by 27% and 23%, while Reordering inflicts steeper drops of 35% and 25%, respectively. Under evasion the effect is far more severe still—Reordering drives LLAGA down by as much as 73% on Cora—confirming that feature perturbations, like structural ones, are most damaging at inference time. GRAPHPROMPTER is consistently the more resilient of the two, and PubMed follows the same qualitative pattern with a somewhat attenuated magnitude.

Crucially, comparing feature and structural perturbations (Figure 3) reveals a fundamental shift in where the vulnerability lies: feature perturbation (specifically Reordering) outperforms the strongest structural attack (METAATTACK) by a wide margin on *both* LLAGA and GRAPHPROMPTER. This demonstrates that graph-aware LLMs weight their input features more heavily than the graph structure, rendering feature-level perturbations the more effective avenue of attack—a marked departure from the structure-dominated vulnerability profile of traditional GNNs. We pursue this observation further by combining the two attack types into a unified attack in the next section.

8 A Unified Imperceptible Attack on Graph-aware LLMs

We propose a unified attack that jointly applies feature and structural perturbations to form a more imperceptible yet more effective adversarial strategy. Our hypothesis is that combining the two perturbation types degrades accuracy more severely than either alone, while keeping the alterations to both structure and features imperceptible.

For the structural component we employ METAATTACK, which subtly adjusts edges, and for the feature component we combine the Homoglyph and Reordering attacks of the previous section. Acting in concert, these perturbations constitute a highly imperceptible attack that simultaneously targets the re-

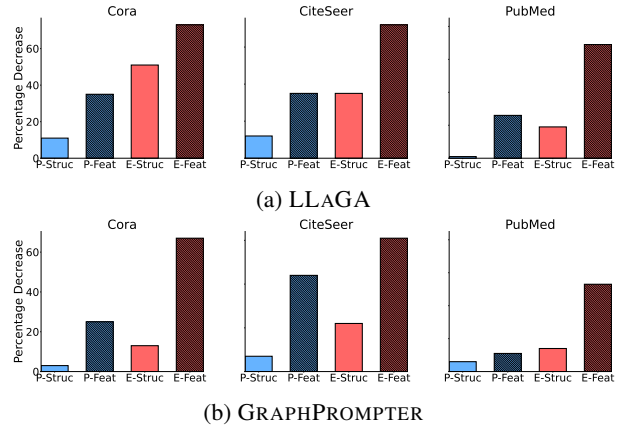


Figure 3: Structure-based versus feature-based adversarial attacks on graph-aware LLMs, with attack success rate measured as the percentage decrease in accuracy relative to clean performance (y-axis). P-Struc, P-Feat, E-Struc, and E-Feat denote poisoning and evasion attacks under structural and feature perturbation, respectively. Feature-based attacks dominate structure-based attacks on both models, confirming that these models are governed more by their textual features than by graph structure.

lational and textual information on which graph-aware LLMs jointly depend.

Results. Table 5 confirms the hypothesis decisively: across all datasets, the unified attack is substantially more effective than either perturbation type in isolation, reducing evasion accuracy by up to 87% and rendering the models effectively unusable. This result crystallizes a central insight into the architecture of graph-aware LLMs: unlike traditional GNNs, whose vulnerability is dominated by structural perturbation, these models are governed by the joint consistency of graph structure *and* node features. The effectiveness of the unified attack thus reflects a fundamental architectural dependence—one that leaves graph-aware LLMs acutely exposed to coordinated, inconspicuous perturbations of both modalities and that directly motivates the dual-pronged defense we develop next.

9 Ablation Study

9.1 Varying Feature Perturbation

To characterize the relationship between perturbation level and attack success, we vary the fraction of perturbed features from 1% to 10%. We report the Reordering attack, whose behavior mirrors that of the Homoglyph attack, and observe a consistent trend across all models and datasets (Cora, CiteSeer, and PubMed): reducing the perturbed fraction monotonically weakens the attack, as expected. Notably, as the perturbation level decreases, the success rates of the poisoning and evasion attacks converge. This convergence follows directly from the diminishing adversarial signal: with few perturbations, the injected signal is too weak to materially affect the model, regardless of whether it is introduced during training (poisoning) or inference (evasion). A detailed plot

Table 4: Accuracy of LLAGA and GRAPH PROMPTER under the imperceptible feature perturbation attacks (Homoglyph and Reordering), for both poisoning and evasion; parentheses give the percentage drop from clean accuracy. Reordering dominates Homoglyph throughout, evasion dominates poisoning, and GRAPH PROMPTER is the more resilient model.

Data	Method	Poisoning		Evasion	
		LLAGA	GRAPH PROMPTER	LLAGA	GRAPH PROMPTER
Cora	Homoglyph	0.65 $\pm$ 0.05 (27%)	0.52 $\pm$ 0.04 (13%)	0.41 $\pm$ 0.04 (54%)	0.38 $\pm$ 0.03 (37%)
	Reordering	0.58 $\pm$ 0.06 (35%)	0.45 $\pm$ 0.05 (25%)	0.24 $\pm$ 0.05 (73%)	0.20 $\pm$ 0.03 (67%)
Citeseer	Homoglyph	0.54 $\pm$ 0.04 (23%)	0.65 $\pm$ 0.04 (7%)	0.32 $\pm$ 0.03 (55%)	0.52 $\pm$ 0.05 (26%)
	Reordering	0.46 $\pm$ 0.03 (35%)	0.39 $\pm$ 0.02 (44%)	0.20 $\pm$ 0.05 (72%)	0.27 $\pm$ 0.02 (61%)
PubMed	Homoglyph	0.72 $\pm$ 0.04 (20%)	0.83 $\pm$ 0.04 (8%)	0.36 $\pm$ 0.06 (60%)	0.55 $\pm$ 0.03 (39%)
	Reordering	0.67 $\pm$ 0.05 (26%)	0.80 $\pm$ 0.03 (11%)	0.28 $\pm$ 0.04 (69%)	0.42 $\pm$ 0.04 (53%)

Table 5: Accuracy under the unified imperceptible attack, which jointly applies structural and feature perturbations; parentheses give the percentage drop from clean accuracy. The unified attack is markedly more effective than either perturbation type alone, reducing evasion accuracy by up to 87%.

Data	Attack	LLAGA	GRAPH PROMPTER
Cora	Poisoning	0.35 $\pm$ 0.05 (61%)	0.28 $\pm$ 0.08 (53%)
	Evasion	0.14 $\pm$ 0.06 (84%)	0.11 $\pm$ 0.05 (82%)
Citeseer	Poisoning	0.26 $\pm$ 0.04 (63%)	0.22 $\pm$ 0.03 (69%)
	Evasion	0.10 $\pm$ 0.09 (86%)	0.13 $\pm$ 0.07 (81%)
PubMed	Poisoning	0.39 $\pm$ 0.07 (57%)	0.32 $\pm$ 0.04 (64%)
	Evasion	0.12 $\pm$ 0.05 (87%)	0.19 $\pm$ 0.04 (79%)

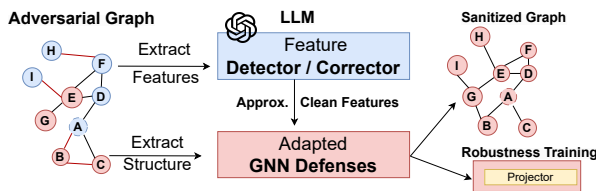


Figure 4: **Overview of the GALGUARD defense.** Perturbed node features and edges are shown in blue and red, respectively. The LLM detects perturbations in the node features extracted from an adversarial graph and produces a corrected version; the refined features, together with the perturbed edges, are then processed by adapted GNN defenses, which either yield a sanitized graph or drive robustness training of the projector. The two stages address feature- and structure-level perturbations respectively, giving GALGUARD its dual-pronged protection.

of these trends is provided in the Appendix (Figure 2).

9.2 Does the Choice of LLM Affect Attack Performance?

We next examine whether the choice of foundation LLM affects adversarial robustness. LLAGA uses Vicuna-7B (Chiang et al. 2023) and GRAPH PROMPTER uses Llama2-7B (Touvron et al. 2023); we replace LLAGA’s backbone with Llama2-7B and compare performance under poisoning and evasion across all datasets. The two backbones attain nearly identical clean accuracy (e.g., Vicuna: 0.89 $\pm$ 0.07; Llama2: 0.90 $\pm$ 0.03 on Cora) and exhibit comparable vulnerability under attack, and the same behavior holds when the backbone is swapped in GRAPH PROMPTER. These results establish that adversarial robustness in this paradigm is governed by the graph-encoding mechanism rather than by the specific foundation LLM—an invariance that indicates our findings are not artifacts of a particular backbone. The full results appear in Table 2 of the Appendix.

10 Defense: GALGUARD

Existing GNN defenses are ill-suited to graph-aware LLMs precisely because of the dual vulnerability our experiments establish—these models are compromised by structural *and* feature perturbations, whereas GNN defenses are designed for the former alone. Many such defenses prune or reweight edges according to node feature similarity (Wu et al. 2019; Dai et al. 2018; Zhang and Zitnik 2020; Jin et al. 2020; Zhang and Ma 2024), and therefore presuppose clean features; once the features are themselves perturbed, these methods fail outright (see Figure 3 in the Appendix). Defenses that decouple structure from features (Shanthamallu, Thiagarajan, and Spanias 2021; Wu et al. 2021), and even OCR-based sanitization (Boucher et al. 2022), likewise break down under coordinated attacks. This dual vulnerability makes a new, integrated defense not merely desirable but necessary.

We therefore introduce the graph-aware LLM defense (GALGUARD), an end-to-end strategy that unifies two complementary components: (1) an LLM-based feature corrector that repairs feature-level perturbations, and (2) adapted GNN-based structural defenses that counter edge-level perturbations. By construction, the two components address the two attack modalities that jointly threaten graph-aware LLMs.

Feature Robustness Strategy. As illustrated in Figure 4, an LLM acts as an adaptive validator that inspects the textual node features for anomalous patterns and issues approximate corrections that restore feature integrity, thereby neutralizing feature-level perturbations. This LLM-based correction offers a practical and readily deployable remedy for the feature vulnerabilities identified above; the specific instructions and prompts are given in Appendix A.6. In our experiments we instantiate the corrector with GPT-4 Turbo.

Graph Structure Robustness Strategy. To harden the model against structural perturbations, we adopt a two-step procedure. First, as a preprocessing step, we apply a *graph purification heuristic* (Wu et al. 2019): for each node v , we compute the cosine similarity between its embedding $\phi(x_v)$ and each neighbor embedding $\phi(x_u)$ and remove the edges of lowest similarity. Second, we integrate *GNNGuard* (Zhang and Zitnik 2020). For GRAPH PROMPTER, we apply GNNGuard’s principles directly to its message-passing layers.

For LLAGA, we propose a novel adaptation that folds the purification step into the construction of the computational tree underlying its node sequence, and we further introduce a learnable *global structural context embedding* (M_{global}) as an additional parameter of LLAGA’s projector. Concatenating M_{global} with the purified node sequence embedding and training the two jointly teaches the model to operate robustly on potentially perturbed graph inputs. Crucially, the plug-and-play design of GALGUARD admits alternative structural defenses—for example, Pro-GNN (Jin et al. 2020)—in place of GNNGuard, so the framework generalizes beyond any single component.

10.1 Results

Table 6 reports accuracy under METAATTACK alongside our two defense configurations: GALGUARD_p, which pairs the feature corrector with the graph purification module, and the full GALGUARD, which additionally incorporates GNNGuard (or its LLAGA-specific adaptation). Across every evaluated dataset (Cora, CiteSeer, and PubMed) and under both poisoning and evasion, our defenses consistently and substantially mitigate METAATTACK. GALGUARD_p already constitutes an effective foundation, demonstrating that jointly correcting feature perturbations with the LLM corrector and applying initial graph purification significantly restores robustness. The full GALGUARD consistently improves upon GALGUARD_p—raising LLAGA’s evasion accuracy on Cora from 0.44 under attack to 0.83, for instance—which quantifies the synergistic benefit of adding the GNN-inspired structural defense on top of feature correction.

Table 6: Accuracy under METAATTACK and under the proposed defense for poisoning and evasion attacks. GALGUARD_p combines feature correction with graph purification, and the full GALGUARD additionally integrates GNNGuard (or its LLAGA-specific adaptation); best results per column are in bold. Both configurations recover accuracy across all datasets and models, and the full GALGUARD is consistently strongest.

Data Method		LLAGA		GRAPHPROMPTER	
		Poisoning	Evasion	Poisoning	Evasion
Cora	METAATTACK	0.79 $\pm$ 0.03	0.44 $\pm$ 0.06	0.58 $\pm$ 0.05	0.52 $\pm$ 0.04
	GALGUARD _p	0.82 $\pm$ 0.05	0.62 $\pm$ 0.04	0.59 $\pm$ 0.03	0.56 $\pm$ 0.04
	GALGUARD	0.85$\pm$0.04	0.83$\pm$0.04	0.60$\pm$0.05	0.59$\pm$0.03
CiteSeer	MetaAttack	0.63 $\pm$ 0.04	0.50 $\pm$ 0.03	0.65 $\pm$ 0.06	0.56 $\pm$ 0.03
	GALGUARD _p	0.64 $\pm$ 0.03	0.56 $\pm$ 0.02	0.67 $\pm$ 0.05	0.60 $\pm$ 0.06
	GALGUARD	0.67$\pm$0.04	0.62$\pm$0.04	0.68$\pm$0.04	0.64$\pm$0.05
PubMed	MetaAttack	0.89 $\pm$ 0.08	0.73 $\pm$ 0.04	0.85 $\pm$ 0.02	0.77 $\pm$ 0.03
	GALGUARD _p	0.90 $\pm$ 0.06	0.78 $\pm$ 0.03	0.85 $\pm$ 0.04	0.81 $\pm$ 0.04
	GALGUARD	0.90$\pm$0.01	0.87$\pm$0.02	0.89$\pm$0.02	0.89$\pm$0.01

11 Conclusion

In this paper, we present the first systematic characterization of the adversarial vulnerabilities of graph-aware LLMs, an area that had remained largely unexplored. Our central

finding is that the mechanism used to integrate graph data into the LLM is itself a decisive source of vulnerability. We uncovered new attack surfaces, showing that design choices such as LLAGA’s node sequence template markedly amplify a model’s susceptibility. We further established that feature perturbation attacks—largely ineffective against traditional GNNs—are highly effective against graph-aware LLMs, frequently outperforming structural attacks, and that a unified attack combining both perturbations can render these models near-useless.

To counter these threats, we proposed GALGUARD, an end-to-end defense that unifies an LLM-based feature corrector with adapted GNN-based structural defenses and consistently restores robustness across datasets and attack types. As graph-aware LLMs are increasingly adopted for high-stakes applications spanning social networks, healthcare, and finance, ensuring their resilience to adversarial exploitation is imperative. We intend this work to serve as a foundation for that effort—one that motivates deeper investigation into the vulnerabilities and defenses of graph-aware LLMs and advances both theoretical understanding and practical safeguards for their real-world deployment.

References

- Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F. L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Ahmedt-Aristizabal, D.; Armin, M. A.; Denman, S.; Fookes, C.; and Petersson, L. 2021. Graph-based deep learning for medical diagnosis and analysis: past, present and future. *Sensors*, 21(14): 4758.
- Anil, R.; Dai, A. M.; Firat, O.; Johnson, M.; Lepikhin, D.; Passos, A.; Shakeri, S.; Taropa, E.; Bailey, P.; Chen, Z.; et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*.
- Bi, B.; Liu, S.; Wang, Y.; Mei, L.; and Chen, X. 2024. Scalable Link Prediction on Large-Scale Heterogeneous Graphs with Large Language Models. *arXiv preprint arXiv:2401.13227*.
- Boucher, N.; Shumailov, I.; Anderson, R.; and Papernot, N. 2022. Bad characters: Imperceptible nlp attacks. In *2022 IEEE Symposium on Security and Privacy (SP)*, 1987–2004. IEEE.
- Brannon, W.; Fulay, S.; Jiang, H.; Kang, W.; Roy, B.; Kabbara, J.; and Roy, D. 2023. Congrat: Self-supervised contrastive pretraining for joint graph and text embeddings. *arXiv preprint arXiv:2305.14321*.
- Chai, Z.; Zhang, T.; Wu, L.; Han, K.; Hu, X.; Huang, X.; and Yang, Y. 2023. GraphLLM: Boosting Graph Reasoning Ability of Large Language Model. *arXiv preprint arXiv:2310.05845*.
- Chandra, S.; Mishra, P.; Yannakoudakis, H.; Nimishakavi, M.; Saeidi, M.; and Shutova, E. 2020. Graph-based Modeling of Online Communities for Fake News Detection. *arXiv preprint arXiv:2008.06274*.

- Chen, R.; Zhao, T.; Jaiswal, A.; Shah, N.; and Wang, Z. 2024. LLaGA: Large Language and Graph Assistant. In *Forty-first International Conference on Machine Learning*.
- Chen, Z.; Mao, H.; Li, H.; Jin, W.; Wen, H.; Wei, X.; Wang, S.; Yin, D.; Fan, W.; Liu, H.; and Tang, J. 2023a. Exploring the Potential of Large Language Models (LLMs) in Learning on Graphs. *arXiv preprint arXiv:2307.03393*.
- Chen, Z.; Mao, H.; Wen, H.; Han, H.; Jin, W.; Zhang, H.; Liu, H.; and Tang, J. 2023b. Label-free Node Classification on Graphs with Large Language Models (LLMs). *arXiv preprint arXiv:2310.04668*.
- Chiang, W.-L.; Li, Z.; Lin, Z.; Sheng, Y.; Wu, Z.; Zhang, H.; Zheng, L.; Zhuang, S.; Zhuang, Y.; Gonzalez, J. E.; Stoica, I.; and Xing, E. P. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality.
- Conti, M.; Li, J.; Picek, S.; and Xu, J. 2022. Label-only membership inference attack against node-level graph neural networks. In *Proceedings of the 15th ACM Workshop on Artificial Intelligence and Security*, 1–12.
- Dai, E.; Lin, M.; Zhang, X.; and Wang, S. 2023. Unnoticeable backdoor attacks on graph neural networks. In *Proceedings of the ACM Web Conference 2023*, 2263–2273.
- Dai, H.; Li, H.; Tian, T.; Huang, X.; Wang, L.; Zhu, J.; and Song, L. 2018. Adversarial attack on graph structured data. In *International conference on machine learning*, 1115–1124. PMLR.
- Dong, N.; Mücke, S.; and Khosla, M. 2022. Mucomid: A multitask graph convolutional learning framework for miRNA-disease association prediction. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 19(6): 3081–3092.
- Duan, K.; Liu, Q.; Chua, T.-S.; Yan, S.; Ooi, W. T.; Xie, Q.; and He, J. 2023. SimTeG: A Frustratingly Simple Approach Improves Textual Graph Learning. *arXiv preprint arXiv:2308.02565*.
- Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Yang, A.; Fan, A.; et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Duddu, V.; Boutet, A.; and Shejwalkar, V. 2020. Quantifying privacy leakage in graph embedding. In *MobiQuitous 2020-17th EAI International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services*, 76–85.
- Fan, W.; Ma, Y.; Li, Q.; He, Y.; Zhao, E.; Tang, J.; and Yin, D. 2019. Graph neural networks for social recommendation. In *The world wide web conference*, 417–426.
- Fatemi, B.; Halcrow, J.; and Perozzi, B. 2023. Talk like a Graph: Encoding Graphs for Large Language Models. *arXiv preprint arXiv:2310.04560*.
- Gong, N. Z.; and Liu, B. 2018. Attribute inference attacks in online social networks. *ACM Transactions on Privacy and Security (TOPS)*, 21(1): 1–30.
- Guo, J.; Du, L.; Liu, H.; Zhou, M.; He, X.; and Han, S. 2023. GPT4Graph: Can Large Language Models Understand Graph Structured Data ? An Empirical Evaluation and Benchmarking. *arXiv preprint arXiv:2305.15066*.
- Hamilton, W.; Ying, Z.; and Leskovec, J. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30.
- He, X.; Bresson, X.; Laurent, T.; Perold, A.; LeCun, Y.; and Hooi, B. 2024. Harnessing Explanations: LLM-to-LM Interpreter for Enhanced Text-Attributed Graph Representation Learning. In *The Twelfth International Conference on Learning Representations*.
- He, X.; Wen, R.; Wu, Y.; Backes, M.; Shen, Y.; and Zhang, Y. 2021. Node-level membership inference attacks against graph neural networks. *arXiv preprint arXiv:2102.05429*.
- Hu, W.; Fey, M.; Zitnik, M.; Dong, Y.; Ren, H.; Liu, B.; Catasta, M.; and Leskovec, J. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems*, 33: 22118–22133.
- Huang, J.; Zhang, X.; Mei, Q.; and Ma, J. 2023a. Can LLMs Effectively Leverage Structural Information for Graph Learning: When and Why. *arXiv preprint arXiv:2309.16595*.
- Huang, X.; Han, K.; Bao, D.; Tao, Q.; Zhang, Z.; Yang, Y.; and Zhu, Q. 2023b. Prompt-based Node Feature Extractor for Few-shot Learning on Text-Attributed Graphs. *arXiv preprint arXiv:2309.02848*.
- Jin, W.; Li, Y.; Xu, H.; Wang, Y.; Ji, S.; Aggarwal, C.; and Tang, J. 2021. Adversarial attacks and defenses on graphs. *ACM SIGKDD Explorations Newsletter*, 22(2): 19–34.
- Jin, W.; Ma, Y.; Liu, X.; Tang, X.; Wang, S.; and Tang, J. 2020. Graph structure learning for robust graph neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 66–74.
- Kipf, T. N.; and Welling, M. 2022. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.
- Kuang, H.; Xu, J.; Zhang, H.; Zhao, Z.; Zhang, Q.; Huang, X.; and Wei, Z. 2023. Unleashing the Power of Language Models in Text-Attributed Graph. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 8429–8441. Association for Computational Linguistics.
- Li, K.; Liu, Y.; Ao, X.; and He, Q. 2023. Revisiting graph adversarial attack and defense from a data distribution perspective. In *The Eleventh International Conference on Learning Representations*.
- Liu, C.; and Wu, B. 2023. Evaluating Large Language Models on Graphs: Performance Insights and Comparative Analysis. *arXiv preprint arXiv:2308.11224*.
- Liu, S.; Nie, W.; Wang, C.; Lu, J.; Qiao, Z.; Liu, L.; Tang, J.; Xiao, C.; and Anandkumar, A. 2023. Multi-modal molecule structure–text model for text-based retrieval and editing. *Nature Machine Intelligence*, 5(12): 1447–1457.
- Liu, Z.; He, X.; Tian, Y.; and Chawla, N. V. 2024. Can we soft prompt LLMs for graph learning tasks? In *Companion Proceedings of the ACM on Web Conference 2024*, 481–484.
- Ma, J.; Ding, S.; and Mei, Q. 2020. Towards more practical adversarial attacks on graph neural networks. *Advances in neural information processing systems*, 33: 4756–4766.

- Mavromatis, C.; Ioannidis, V. N.; Wang, S.; Zheng, D.; Adeshina, S.; Ma, J.; Zhao, H.; Faloutsos, C.; and Karypis, G. 2023. Train your own GNN teacher: Graph-aware distillation on textual graphs. *arXiv preprint arXiv:2304.10668*.
- Olatunji, I. E.; Hizber, A.; Sihlovec, O.; and Khosla, M. 2023a. Does black-box attribute inference attacks on graph neural networks constitute privacy risk? *arXiv preprint arXiv:2306.00578*.
- Olatunji, I. E.; Nejd, W.; and Khosla, M. 2021. Membership inference attack on graph neural networks. In *2021 Third IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*, 11–20. IEEE.
- Olatunji, I. E.; Rathee, M.; Funke, T.; and Khosla, M. 2023b. Private Graph Extraction via Feature Explanations. *Proceedings on Privacy Enhancing Technologies*.
- Pan, S.; Luo, L.; Wang, Y.; Chen, C.; Wang, J.; and Wu, X. 2024. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*.
- Peng, Y.; Lin, S.; Chen, Q.; Xu, L.; Ren, X.; Li, Y.; and Xu, J. 2024. ChatGraph: Chat with Your Graphs. *arXiv preprint arXiv:2401.12672*.
- Qin, Y.; Wang, X.; Zhang, Z.; and Zhu, W. 2023. Disentangled Representation Learning with Large Language Models for Text-Attributed Graphs. *arXiv preprint arXiv:2310.18152*.
- Shanthamallu, U. S.; Thiagarajan, J. J.; and Spanias, A. 2021. Uncertainty-matching graph neural networks to defend against poisoning attacks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, 9524–9532.
- Storn, R.; and Price, K. 1997. Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *Journal of global optimization*, 11: 341–359.
- Su, B.; Du, D.; Yang, Z.; Zhou, Y.; Li, J.; Rao, A.; Sun, H.; Lu, Z.; and Wen, J.-R. 2022. A molecular multimodal foundation model associating molecule graphs with natural language. *arXiv preprint arXiv:2209.05481*.
- Sun, L.; Dou, Y.; Yang, C.; Zhang, K.; Wang, J.; Philip, S. Y.; He, L.; and Li, B. 2022. Adversarial attack and defense on graph data: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(8): 7693–7711.
- Sun, S.; Ren, Y.; Ma, C.; and Zhang, X. 2023. Large Language Models as Topological Structure Enhancers for Text-Attributed Graphs. *arXiv preprint arXiv:2311.14324*.
- Tan, Y.; Zhou, Z.; Lv, H.; Liu, W.; and Yang, C. 2023. WalkLM: A Uniform Language Model Fine-tuning Framework for Attributed Graph Embedding. In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Team, G.; Anil, R.; Borgeaud, S.; Wu, Y.; Alayrac, J.-B.; Yu, J.; Soricut, R.; Schalkwyk, J.; Dai, A. M.; Hauth, A.; et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.
- Tian, Y.; Song, H.; Wang, Z.; Wang, H.; Hu, Z.; Wang, F.; Chawla, N. V.; and Xu, P. 2024. Graph neural prompting with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 19080–19088.
- Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; and Bengio, Y. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.
- Wang, J.; Zhang, S.; Xiao, Y.; and Song, R. 2021. A review on graph neural network methods in financial applications. *arXiv preprint arXiv:2111.15367*.
- Wang, X.; and Wang, W. H. 2022. Group property inference attacks against graph neural networks. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2871–2884.
- Wei, Y.; Fu, S.; Jiang, W.; Kwok, J. T.; and Zhang, Y. 2024a. Rendering graphs for graph reasoning in multimodal large language models. *arXiv preprint arXiv:2402.02130*.
- Wei, Y.; Huang, Q.; Kwok, J. T.; and Zhang, Y. 2024b. Kicgpt: Large language model with knowledge in context for knowledge graph completion. *arXiv preprint arXiv:2402.02389*.
- Wenkel, F.; Wolf, G.; and Knyazev, B. 2023. Pretrained Language Models to Solve Graph Tasks in Natural Language. In *ICML workshop on Structured Probabilistic Inference & Generative Modeling*.
- Wu, H.; Wang, C.; Tyshetskiy, Y.; Docherty, A.; Lu, K.; and Zhu, L. 2019. Adversarial examples on graph data: Deep insights into attack and defense. *arXiv preprint arXiv:1903.01610*.
- Wu, X.; Wu, H.; Zhou, X.; and Lu, K. 2021. CoG: a Two-View Co-training Framework for Defending Adversarial Attacks on Graph. *arXiv preprint arXiv:2109.05558*.
- Xie, H.; Zheng, D.; Ma, J.; Zhang, H.; Ioannidis, V. N.; Song, X.; Ping, Q.; Wang, S.; Yang, C.; Xu, Y.; et al. 2023. Graph-aware language model pre-training on a large graph corpus can help multiple graph applications. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 5270–5281.
- Yang, J.; Liu, Z.; Xiao, S.; Li, C.; Lian, D.; Agrawal, S.; Singh, A.; Sun, G.; and Xie, X. 2021. GraphFormers: GNN-nested Transformers for Representation Learning on Textual graph. In *Proceedings of the 35th Conference on Neural Information Processing Systems*.
- Yang, Z.; Cohen, W.; and Salakhudinov, R. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*, 40–48. PMLR.
- Ye, R.; Zhang, C.; Wang, R.; Xu, S.; and Zhang, Y. 2023. Natural Language is All a Graph Needs. *arXiv preprint arXiv:2308.07134*.
- Zhang, A.; and Ma, J. 2024. Defensevgae: Defending against adversarial attacks on graph data via a variational graph autoencoder. In *International Conference on Intelligent Computing*, 313–324. Springer.
- Zhang, J. 2023. Graph-toolformer: To empower llms with graph reasoning ability via prompt augmented by chatgpt. *arXiv preprint arXiv:2304.11116*.

- Zhang, X.; and Zitnik, M. 2020. GnnGuard: Defending graph neural networks against adversarial attacks. *Advances in neural information processing systems*, 33: 9263–9275.
- Zhang, Z.; Chen, M.; Backes, M.; Shen, Y.; and Zhang, Y. 2022. Inference attacks against graph neural networks. In *31st USENIX Security Symposium (USENIX Security 22)*, 4543–4560.
- Zhang, Z.; Liu, Q.; Huang, Z.; Wang, H.; Lu, C.; Liu, C.; and Chen, E. 2021. Graphmi: Extracting private graph data from graph neural networks. *arXiv preprint arXiv:2106.02820*.
- Zhang, Z.; Wang, X.; Zhang, Z.; Li, H.; Qin, Y.; and Zhu, W. 2023. LLM4DyG: Can Large Language Models Solve Problems on Dynamic Graphs? *arXiv preprint arXiv:2310.17110*.
- Zhao, J.; Zhuo, L.; Shen, Y.; Qu, M.; Liu, K.; Bronstein, M.; Zhu, Z.; and Tang, J. 2023. GraphText: Graph Reasoning in Text Space. *arXiv preprint arXiv:2310.01089*.
- Zhu, Y.; Wang, Y.; Shi, H.; and Tang, S. 2024. Efficient tuning and inference for large language models on textual graphs. *arXiv preprint arXiv:2401.15569*.
- Zügner, D.; Akbarnejad, A.; and Günnemann, S. 2018. Adversarial attacks on neural networks for graph data. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2847–2856.
- Zügner, D.; and Günnemann, S. 2019. Adversarial Attacks on Graph Neural Networks via Meta Learning. In *International Conference on Learning Representations (ICLR)*.

A Appendix

Organization: The Appendix is organized as follows. Appendix A.1 presents a detailed taxonomy of graph-aware LLMs, and Appendix A.2 reviews related work on attacks against Graph Neural Networks (GNNs). Appendix A.3 gives an overview of the adversarial attacks NETTACK and METAATTACK, while Appendix A.4 details the algorithm for the node sequence injection attack on LLAGA. Appendix A.5 reports our ablation studies, including the effect of varying the fraction of perturbed features and of changing the base LLM on attack performance. Finally, Appendix A.7 provides examples of the feature attacks together with the template for the LLM corrector, and Appendix A.8 discusses the broader implications of our findings.

Code. Our code will be released upon acceptance.

A.1 Graph Encoding-Based Taxonomy of LLM Adaptations

The integration of large language models (LLMs) with graph data is a rapidly emerging research area, with a growing body of methods adapting or augmenting LLMs for graph-related tasks such as reasoning, representation, and inference. These methods divide cleanly into two categories according to how they encode graphs for LLMs: *textual descriptions of graphs*, which express relational information as natural language, and *learned graph projectors*, which employ a trained projector to map graph data into embeddings or structured forms that the LLM can consume directly. In this section we describe the methods within each category. Figure 5 illustrates both approaches, and Table 7 summarizes the distinctions between them.

Graph Representation as a Textual Description In this first category, graph data is expressed as natural language, allowing LLMs to process graph structures as textual descriptions. The approach has been explored extensively, with works such as (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a) converting graph data into textual formats, typically paired with task-specific querying instructions. For instance, Tan et al. (2023) introduced a unified framework for attributed graph embedding that generates random-walk-based textual sequences from graphs and uses them to fine-tune LLMs. Similarly, Zhao et al. (2023) addressed graph reasoning within text spaces, translating graph structures into natural language to facilitate LLM interaction, while Wei et al. (2024a) extended this idea by augmenting textual graph representations with visual ones, strengthening the reasoning capabilities of LLMs on graph data. In multimodal settings, Liu et al. (2023) and Su et al. (2022) aligned molecular graphs with textual descriptions, enabling advances in drug discovery and molecular retrieval. Furthermore, Huang et al. (2023a) examined how incorporating graph structure into LLM prompts affects predictive performance, finding that LLMs tend to process graph-related prompts as contextual paragraphs rather than explicitly recognizing graph structures—a result that underscores the decisive role of prompt design, as further explored in (Brannon et al.

2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024), which extended LLMs to heterogeneous and large-scale graph tasks. Complementing these efforts, (He et al. 2024) employed LLM-generated explanations as features to improve both the performance and the interpretability of LLMs on graphs.

While still representing graphs as textual descriptions, a further line of work couples pretraining strategies with GNNs to strengthen the graph reasoning of LLMs. For example, Chen et al. (2023a) proposed two LLM–GNN integration strategies: (1) using LLMs to enrich node features for node classification, and (2) employing LLMs as direct predictors for graph tasks. In the same spirit, Chandra et al. (2020) used LLMs as text encoders and GNNs as graph-structure encoders for tasks such as fake-news detection in online communities; Chen et al. (2023b) performed label-free node classification by using LLMs to annotate nodes that then supervise a GNN; and Mavromatis et al. (2023) employed an LLM as a feature extractor for GNNs.

For inference, Zhu et al. (2024) proposed a fine-tuning method for textual graphs that combines LLMs and GNNs through a tunable side structure attached to each LLM layer, improving both training efficiency and inference speed. Similarly, Duan et al. (2023) introduced SimTeG, a simple yet effective approach for textual graph learning in which the LLM is first fine-tuned on a downstream task, such as node classification, after which node embeddings are obtained from the last hidden states of the fine-tuned model. Additionally, Zhang (2023) proposed Graph-toolformer, which uses ChatGPT-powered prompt engineering to augment the graph reasoning of LLMs.

Although these approaches broaden the accessibility of LLMs for graph-related tasks, they struggle to fully capture the structural intricacies inherent in graph data, and their performance remains heavily contingent on effective prompt engineering—a process that demands careful, iterative design to reach optimal results.

Graph Representation as a Learned Projector To overcome the challenges of textual-representation approaches, learned-projector methods transform graph data into embeddings or graph vectors that the LLM can consume more directly. For instance, Chen et al. (2024) introduced LLAGA, a large language and graph assistant that preserves the general-purpose nature of the LLM while converting graph structures into graph tokens suitable for LLM processing. Similarly, (Liu et al. 2024; Qin et al. 2023) injected graph-structure information into a pre-trained LLM through tailored disentangled graph neural network layers. Tian et al. (2024) proposed graph neural prompts that encode knowledge graphs into graph embeddings the LLM can leverage at inference, while Huang et al. (2023b) introduced a prompt-based node feature extractor (G-Prompt) for few-shot learning on text-attributed graphs, combining a graph adapter (a learnable GNN layer) with task-specific prompts to capture neighborhood information and extract informative node features.

A further promising line of work condenses graph information into fixed-length prefixes via graph transformers attached to each LLM layer, thereby enhancing its reasoning (Chai

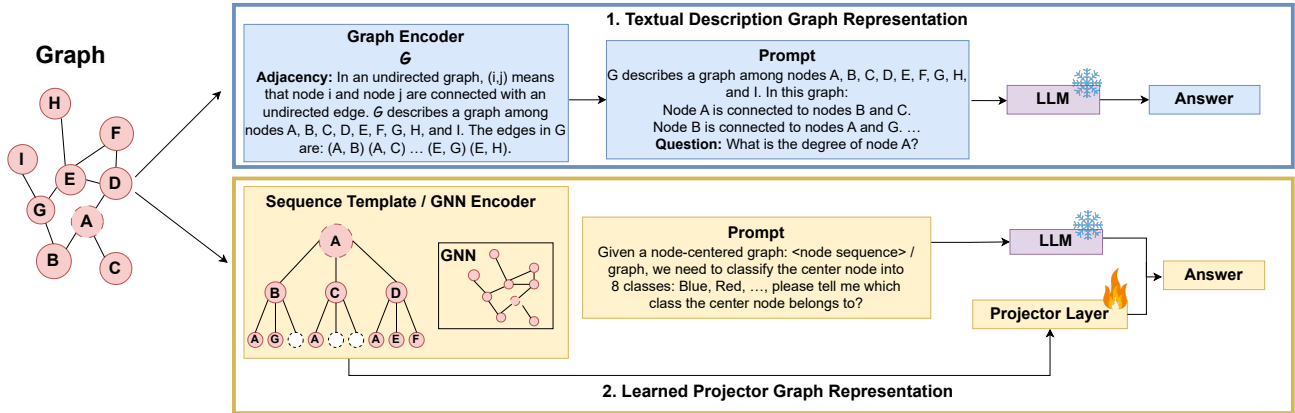


Figure 5: **Graph encoding-based adaptations of LLMs.** Under the textual-description representation, graphs are rendered as natural language (top). Under the learned-projector representation, graphs are encoded either by node templates that serialize the graph into sequences or by GNNs (bottom). In both cases the LLM remains frozen; only the projector that maps the graph into tokens is trained.

Table 7: Comparison of the two graph encoding-based adaptations of LLMs, contrasting the textual-description and learned-projector approaches along graph encoding, LLM modification, structural representation, flexibility, computational cost, and suitability for graph tasks.

Aspect	Textual Description Approach	Learned Projector Approach
Graph Encoding	Converts graph data (nodes, edges) into natural language descriptions.	Uses a learned projector to encode graph structures into embeddings or graph tokens.
LLM Modifications	No modification to the LLM.	No modification to the LLM.
Graph Structure Representation	Describes graph topology and relationships using textual narratives.	Encodes relational and structural information directly into vector representations.
Flexibility	Can only be used for simple graphs.	More flexible in representing complex graph structures.
Computational Cost	No additional training is required. Lower cost.	Lightweight projector is trained, typically a linear layer. Minimal cost.
Suitability for Graph Tasks	May lose some relational nuance due to limitations in textual descriptions.	Captures detailed structural information, improving performance on graph-specific tasks.

et al. 2023). Relatedly, Yang et al. (2021) developed GraphFormers, which integrates text encoding and graph aggregation in an iterative workflow by nesting GNNs alongside the transformer blocks of a language model, and Peng et al. (2024) developed ChatGraph, an interactive interface that lets users query their graphs in natural language by representing graphs as sequential paths and fine-tuning the LLM on the graph data for downstream analysis.

We deliberately exclude projector adaptations that require retraining the LLM, since they inflate computational cost substantially without commensurate performance gains, and instead concentrate on adaptations that operate on **frozen pre-trained LLMs** (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). This choice not only reduces cost but also affords greater flexibility across LLM backbones, letting us exploit the extensive knowledge and capabilities of LLMs—without the overhead of retraining—while still adapting them to specific graph tasks. Throughout this work we employ two representative and complementary projector methods: **LLAGA** (Chen et al. 2024), which pairs node-level templates with a linear projector, and **GRAPHPROMPTER** (Liu et al. 2024), which employs a GNN-based projector to encode the graph.

This work. We likewise exclude textual-description methods, for two reasons. First, they struggle to capture the structural intricacies inherent in graph data: they are generally confined to simple tasks such as describing basic graph properties like node degree and become infeasible for larger graphs, since the prompt token limit cannot accommodate the detailed descriptions large structures demand and is quickly exhausted. Second, and decisively for our purposes, textual-description methods introduce no unique attack surface: because the graph is rendered as plain text (e.g., "Node A is connected to Node B"), they are exposed only to conventional text-based adversarial attacks rather than the structural or feature perturbations specific to graph-based models. They therefore lack the complexity that would expose them to the novel, graph-specific vulnerabilities that are the focus of our investigation into the adversarial robustness of graph-aware LLMs.

A.2 Attacks on GNN

GNNs are subject to a broad spectrum of attacks: membership inference attacks (Olatunji, Nejd, and Khosla 2021; He et al. 2021; Conti et al. 2022), which determine whether a specific node belonged to the training set; attribute inference attacks (Gong and Liu 2018; Olatunji et al. 2023a; Duddu, Boutet, and Shejwalkar 2020), which exploit the GNN’s outputs and learned embeddings to recover sensitive or missing node attributes; property inference attacks (Zhang et al. 2022; Wang and Wang 2022), which extract global properties of the entire graph; adversarial attacks (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Li et al. 2023; Dai et al. 2018; Sun et al. 2022), which degrade predictive performance through subtle perturbations to the graph or node features; and graph reconstruction attacks (Zhang et al. 2021; Olatunji et al. 2023b), which recover the underlying graph structure from learned embeddings or model outputs.

Among these, adversarial attacks are of particular concern

owing to their direct impact on predictive performance. Such attacks introduce small, carefully crafted *perturbations* into the graph—modifying node features or altering edges—so as to substantially degrade the GNN. They fall into two types: **poisoning attacks**, which act during the *training phase* by manipulating the training data, and **evasion attacks**, which act at *test time* by perturbing the input graph to mislead the model’s predictions. For instance, Zügner, Akbarnejad, and Günnemann (2018) showed that adversarial perturbations of graph structure or node features can substantially degrade GNN performance; their method, **NETTACK**, uses greedy optimization to compute the minimal perturbations that maximize the training loss, thereby identifying the smallest change in features or structure sufficient to disrupt the model’s predictions. Li et al. (2023) revisited graph adversarial attacks from a data-distribution perspective, showing that adversarial examples can substantially shift the underlying data distribution and proposing a corresponding defense. Complementary, Wu et al. (2019) provided a comprehensive analysis of adversarial examples on graph data, examining the vulnerability of GNNs to small perturbations and proposing defenses that detect and counteract adversarial modifications to enhance robustness. **METAATTACK** (Zügner and Günnemann 2019) conducts both poisoning and evasion attacks by treating the graph structure as an optimizable hyperparameter, modifying it directly via gradient descent within a bi-level optimization formulation. It employs meta-gradients—which capture how small structural perturbations affect the attacker’s loss after training—to make precise, performance-degrading updates to the graph.

In this paper we adopt **NETTACK** (Zügner, Akbarnejad, and Günnemann 2018) and **METAATTACK** (Zügner and Günnemann 2019) as representative adversarial attacks spanning both poisoning and evasion strategies: poisoning attacks compromise model integrity during training, whereas evasion attacks degrade performance at inference (test) time.

A.3 How NETTACK and METAATTACK Work

NETTACK perturbs the graph structure by adding and removing edges while preserving the degree distribution of the perturbed graph. It uses greedy optimization to compute the minimal perturbations that maximize the training loss with respect to a surrogate model—a linearized two-layer graph convolutional network (GCN) that approximates how structural changes affect classification. The objective is to raise the model’s confidence in incorrect predictions by maximizing the distance between the logits on the perturbed and clean graphs. Concretely, **NETTACK** enumerates candidate edges around a target node and evaluates, for each incident edge, a score quantifying how strongly altering that edge would affect the target node’s classification; it then selects the highest-scoring edge and adds or removes it accordingly. Because **NETTACK** is a targeted attack—it induces misclassification of a single specified node—we follow (Zügner and Günnemann 2019) to convert it into an untargeted evaluation that instead compromises overall node classification: we randomly select a test node, attack it with **NETTACK** while considering all nodes in the network, and aggregate the results over the test nodes.

METAATTACK optimizes the graph structure directly via gradient descent, treating the adjacency matrix as a hyperparameter. It frames the attack as a bi-level optimization problem in which the inner level minimizes the training loss on the original graph and the outer level minimizes the loss on the poisoned graph. Like **NETTACK**, it relies on a linearized two-layer GCN surrogate to approximate how structural changes affect performance. A score function evaluates the impact of each candidate perturbation, and the attack flips the sign of the meta-gradients (gradients with respect to the hyperparameters) for connected nodes to indicate which edges to remove; it then greedily selects the highest-scoring perturbations—adding or removing edges—to maximize its impact.

For both methods we impose the unnoticeability constraint of (Zügner, Akbarnejad, and Günnemann 2018), which permits only slight changes to the graph’s degree distribution; accordingly, we perturb only 10% of the edges.

For poisoning **LLAGA**, we use the perturbed graph structure produced by **NETTACK** or **METAATTACK** to generate the node-level sequences and retrain the projector that maps those sequences into token embeddings. For poisoning **GRAPH-PROMPTER**, we use the same perturbed structure, re-encode the graph with a graph attention network (GAT), and retrain the projector on the perturbed graph. For evasion, the graph-aware LLM is trained on unperturbed data, and perturbations are introduced only at inference.

A.4 Algorithm for Node Sequence Template Injection Attacks

Algorithm 1 specifies the node sequence template injection attack on **LLAGA**. The model constructs a *fixed-shape* computational tree for each node u , sampling k neighbors per node up to depth d (lines 1–14); when a node has fewer than k neighbors, placeholders fill the remaining slots, and precisely these placeholders constitute the attack surface. By manipulating the graph, an attacker injects adversarially chosen nodes into the placeholder positions. We instantiate three strategies of increasing severity: *Non-Adjacent Injection (NI)* inserts nodes lying outside the 2-hop neighborhood of u (lines 21–27); *Supernode Injection (SI)* injects a single high-degree node into one placeholder (lines 29–33); and *Multiple Supernode Injection (MSI)* distributes distinct high-degree nodes across all placeholders (lines 35–43). In every case, the injected uninformative nodes corrupt the target node’s sequence representation, which in turn impairs training of the projection layer and degrades performance.

A.5 Ablation Studies

We now present the full results for the ablation studies summarized in the main paper.

Ablation Results: Varying Feature Perturbation Figure 6 reports the performance of the two graph-aware LLMs, **LLAGA** and **GRAPH-PROMPTER**, as the fraction of perturbed features is varied. Increasing the perturbed fraction consistently degrades accuracy for both models, confirming that both are markedly sensitive to feature-level attacks and that

Algorithm 1: Injection Attacks on Neighborhood Detail Template for Obtaining Node Sequence.

Input: Graph $G = (V, E)$, target node u , tree depth d , children size k

Output: Modified graph G' with injected nodes

```

1: procedure NEIGHBORHOODDETAILTEMPLATE( $u, d, k$ )
2:    $T \leftarrow \emptyset$  ▷ Initialize empty tree
3:    $T.\text{root} \leftarrow u$ 
4:   for level  $\ell = 1$  to  $d$  do
5:     for each node  $v$  at level  $\ell - 1$  do
6:        $N_v \leftarrow \text{Sample}(\mathcal{N}(v), k)$  ▷ Sample  $k$  neighbors
7:       if  $|N_v| < k$  then
8:         Fill remaining slots with placeholders
9:       end if
10:      Add  $N_v$  as children of  $v$  in  $T$ 
11:    end for
12:  end for
13:  return  $T$ 
14: end procedure

15: procedure ATTACK( $G, u, d, k, \text{AttackStrategy}$ )
16:   $G' \leftarrow G$  ▷ Create copy of original graph
17:   $T \leftarrow \text{NEIGHBORHOODDETAILTEMPLATE}(u, d, k)$ 
18:   $P \leftarrow \text{GetPlaceholders}(T)$ 
19:  if  $|P| > 0$  then
20:     $\mathcal{N}_2 \leftarrow \text{2-hop neighborhood of } u$ 
21:    if AttackStrategy = NI then
22:       $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
23:       $V_{\text{inject}} \leftarrow \text{Sample}(V_{\text{non-adjacent}}, |P|)$ 
24:      for each placeholder position  $p \in P$  do
25:         $v \leftarrow \text{Next node from } V_{\text{inject}}$ 
26:        Add edge  $(\text{parent}(p), v)$  to  $G'$ 
27:      end for
28:    else if AttackStrategy = SI then
29:       $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
30:       $s \leftarrow \text{GetHighestDegreeNode}(V_{\text{non-adjacent}})$ 
31:      for first placeholder position  $p \in P$  do
32:        Add edge  $(\text{parent}(p), s)$  to  $G'$ 
33:        break ▷ Only use first placeholder
34:      end for
35:    else if AttackStrategy = MSI then
36:       $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
37:       $S \leftarrow \text{GetNodesOrderedByDegree}(V_{\text{non-adjacent}})$ 
38:      for  $i = 1$  to  $|P|$  do
39:         $s \leftarrow \text{Get } i\text{-th node from } S$ 
40:         $p \leftarrow \text{Get } i\text{-th placeholder from } P$ 
41:        Add edge  $(\text{parent}(p), s)$  to  $G'$ 
42:      end for
43:    end if
44:  end if
45:  return  $G'$ 
46: end procedure

```

the degradation scales monotonically with the strength of the perturbation.

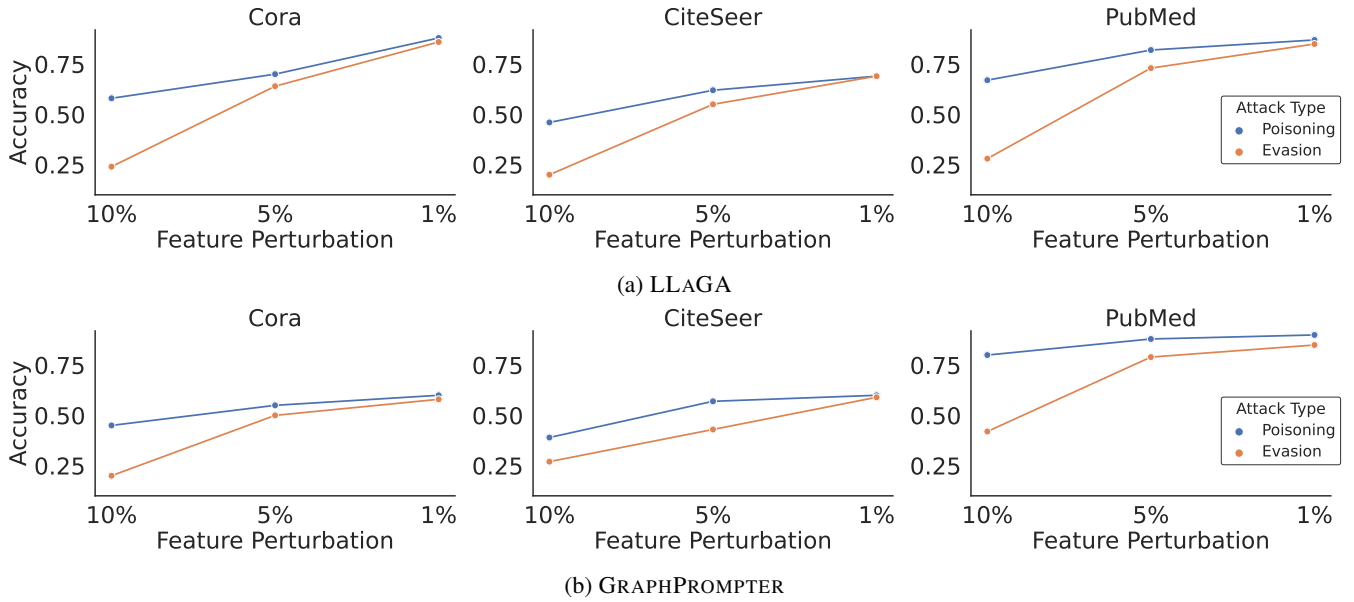


Figure 6: Performance of the graph-aware LLMs as the percentage of perturbed features is varied; from an attacker’s perspective, lower accuracy indicates a more successful attack. Accuracy declines monotonically with the perturbed fraction for both models.

Ablation Results: Does the Choice of LLM Affect Attack Performance? Table 8 shows that the choice of base LLM has negligible impact on both clean performance and adversarial vulnerability within the LLAGA framework. Across Cora, Citeseer, and PubMed, the Vicuna-7B and Llama2-7B backbones attain nearly identical clean accuracy and degrade comparably under both poisoning and evasion. Under METAATTACK evasion on Cora, for example, accuracy falls to 0.44 ± 0.06 for Vicuna-7B and 0.43 ± 0.08 for Llama2-7B—a difference well within one standard deviation—and the same pattern holds across all datasets and attack types. Attack effectiveness in this paradigm is thus governed by the graph-encoding mechanism rather than by the specific foundation LLM.

A.6 Results on the ArXiv Dataset

We further evaluate the node sequence template attacks on the large-scale ArXiv dataset—169,343 nodes, 1,166,243 edges, sparsity 0.81—to establish their effectiveness on large, real-world graphs. As reported in Table 9, ArXiv exhibits a pronounced gap between the two regimes: poisoning consistently outperforms evasion across all three injection strategies, reversing the ordering observed on Cora, Citeseer, and PubMed. This reversal points to structural characteristics that render ArXiv more vulnerable to poisoning yet more robust to evasion; notably, despite the graph’s very low density, injecting even a single supernode can substantially degrade performance.

A.7 Feature Attack Examples and Defense Template

Feature Attack Examples The prompt examples in Appendix A.7 illustrate our adversarial attacks on real features

from the Cora dataset. The “input” is the text received by the LLM once the attack has been applied. The essential point is that, although a human reader perceives the original and perturbed text as visually identical or nearly so, the LLM ingests a different sequence of Unicode characters or a different character order, which can alter its behavior. We illustrate this in Table 10 for the homoglyph attack and Table 11 for the reordering attack.

Feature Corrector Defense Template We designed a feature corrector module to counter imperceptible attacks—specifically homoglyph substitutions and character reorderings—implemented as a prompt-based defense over LLMs. The prompt is carefully constructed to cast the LLM as a linguistic validator whose objective is to detect and repair subtle, human-imperceptible perturbations.

The prompt template is shown below. By explicitly specifying the task and prescribing a clear output format, it substantially improves the LLM’s corrective performance.

A.8 Detailed Discussion

Our findings yield several critical insights into the vulnerabilities and design considerations of graph-aware LLMs. As the integration of graph-structured data into LLMs continues to gain traction, it is essential to examine not only the performance benefits of these novel architectures but also their vulnerabilities. Our systematic investigation of adversarial attacks on two representative graph-aware LLMs—LLAGA and GRAPHPROMPTER—exposes significant weaknesses, particularly under the poisoning and evasion attacks traditionally used against GNNs. We discuss the key observations below.

Impact of Graph Encoding on Adversarial Vulnerability The central observation from our results is that the mech-

Table 8: Effect of the base LLM on clean performance and on robustness under different attacks, shown for LLAGA with Vicuna-7B versus Llama2-7B backbones. Accuracy is nearly identical across backbones in every setting, demonstrating that susceptibility to attack is essentially independent of the base model.

Data	Base Model	Clean	Poisoning			Evasion		
			NETTACK	METAATTACK	MSI	NETTACK	METAATTACK	MSI
Cora	Vicuna-7B	0.89 ± 0.07	0.87 ± 0.04	0.79 ± 0.03	0.82 ± 0.04	0.55 ± 0.09	0.44 ± 0.06	0.30 ± 0.05
	Llama2-7B	0.90 ± 0.03	0.86 ± 0.02	0.77 ± 0.05	0.82 ± 0.02	0.56 ± 0.07	0.43 ± 0.08	0.31 ± 0.01
Citeseer	Vicuna-7B	0.71 ± 0.04	0.64 ± 0.05	0.63 ± 0.04	0.62 ± 0.02	0.59 ± 0.04	0.50 ± 0.03	0.24 ± 0.05
	Llama2-7B	0.71 ± 0.05	0.65 ± 0.03	0.60 ± 0.03	0.61 ± 0.04	0.58 ± 0.03	0.48 ± 0.05	0.26 ± 0.03
PubMed	Vicuna-7B	0.90 ± 0.04	0.89 ± 0.03	0.89 ± 0.08	0.81 ± 0.03	0.84 ± 0.05	0.73 ± 0.04	0.76 ± 0.03
	Llama2-7B	0.91 ± 0.06	0.88 ± 0.04	0.90 ± 0.02	0.80 ± 0.04	0.85 ± 0.03	0.75 ± 0.02	0.77 ± 0.02

Prompt for Feature Corrector	
You are a linguistic validator. Your task is to detect and correct any subtle, imperceptible text perturbations such as homoglyph substitutions (e.g., replacing "o" with "0") and character reorderings. These changes are often used to attack machine learning models. Given a text string, carefully analyze it. If you identify any perturbations, provide the corrected, original-looking version of the text. If no perturbations are found, return the original text unchanged.	
Example 1 (Homoglyph Substitution): Input: Title: The mgprir huristic fr discvring prtin squnc pttrens Abstract: Svrl cmputr lgrithms fr discvring pttrens in grups f prttn squncs r in us tht r bsd on fitting th prmtrs f sttistical mdl to grup of rlted squncs. Output: Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.	
Example 2 (Character Reordering): Input: Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dscribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiottn of the apclpiation of three machine larening algorithms-lR, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae. Output: Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-lR, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.	
Example 3 (Mixed Homoglyph & Reordering): Input: Title: Submitted to NIPS96, Sctin: Applications. Prfrnc: Oral prsentatin Reinforcement Lrning for Dynmic Channl Allcation in Abstract: In cllulr tlphne systems, n imrtnt prblm is to dynmiclly alloct th cmmunictin rsurc (chnnls) so as to mximize srvc in stchstic cllr nvirnmt. This prblm is naturally frmultd s dynmic prgrmming prblm and we us a reinforcement lerning (RL) mthd to find dynmic chnnl llction plicies that ar bttr than prvius heuristic slutins. Output: Title: Submitted to NIPS96, Section: Applications. Preference: Oral presentation Reinforcement Learning for Dynamic Channel Allocation in Abstract: In cellular telephone systems, an important problem is to dynamically allocate the communication resource (channels) so as to maximize service in a stochastic caller environment. This problem is naturally formulated as a dynamic programming problem and we use a reinforcement learning (RL) method to find dynamic channel allocation policies that are better than previous heuristic solutions.	
Your Task: Text to correct: {perturbed.text.here}	
Corrected Text:	

Figure 7: The prompt template used for the LLM-based feature corrector module, which explicitly defines the LLM’s role, its task, and the expected output format.

Table 9: Accuracy of LLAGA under the node sequence template injection attacks, extended to the large-scale ArXiv dataset. *NI*, *SI*, and *MSI* denote the non-adjacent, single-supernode, and multiple-supernode injection attacks, respectively; Clean reports unattacked performance. On ArXiv, uniquely, poisoning is more damaging than evasion across all three strategies.

Data	Attack	NI	SI	MSI
Cora	Clean	————	0.89 ± 0.07	————
	Poisoning	0.86 ± 0.03	0.84 ± 0.06	0.82 ± 0.04
	Evasion	0.42 ± 0.08	0.36 ± 0.05	0.30 ± 0.05
Citeseer	Clean	————	0.71 ± 0.04	————
	Poisoning	0.69 ± 0.05	0.66 ± 0.03	0.62 ± 0.02
	Evasion	0.36 ± 0.06	0.30 ± 0.04	0.24 ± 0.05
PubMed	Clean	————	0.90 ± 0.04	————
	Poisoning	0.88 ± 0.04	0.85 ± 0.05	0.81 ± 0.03
	Evasion	0.85 ± 0.07	0.82 ± 0.04	0.76 ± 0.03
ArXiv	Clean	————	0.73 ± 0.02	————
	Poisoning	0.50 ± 0.03	0.43 ± 0.04	0.42 ± 0.03
	Evasion	0.71 ± 0.06	0.69 ± 0.02	0.60 ± 0.03

anism used to integrate graph structure into an LLM can inadvertently heighten its susceptibility to adversarial attack. Specifically, LLAGA’s node sequence template creates an exploitable vulnerability that yields substantial performance degradation. This attack surface, unique to LLAGA, exposes a critical tradeoff in graph-aware LLM design: although node-level templates provide an effective means of incorporating graph structure and achieve strong performance on graph tasks, they simultaneously open new attack vectors absent from traditional GNNs. GRAPH-PROMPTER, which encodes graphs through a GNN-based projector, is comparatively more resilient to these attacks—underscoring the potential of hybrid architectures that combine the complementary strengths of LLMs and GNNs.

Sensitivity of Graph-aware LLMs to Feature Perturbations Beyond the vulnerabilities of the graph-encoding mechanisms, our results establish the striking effectiveness of feature perturbation attacks across both graph-aware LLMs. Despite their differing encodings, LLAGA and GRAPH-PROMPTER are both acutely sensitive to imperceptible perturbations of the node features. This reflects a broader tension in the LLM paradigm: although these models excel at processing textual data, their robustness to adversarial perturbations—especially those targeting node features—remains a critical open challenge. That an adversary can degrade performance by subtly and undetectably altering node attributes calls the integrity of graph-aware LLMs into question, particularly in security-sensitive applications. While text sanitization techniques such as those discussed by (Boucher et al. 2022) offer partial defenses, countering imperceptible manipulations like homograph substitution remains difficult. Our proposed use of an LLM as a corrector—prompting the model to detect and repair suspicious or malformed inputs before downstream processing—provides an effective and

practical route to mitigating this vulnerability.

Data-dependent Variability of Poisoning and Evasion Attacks on Graph-aware LLMs Our findings show that the efficacy of poisoning and evasion attacks against graph-aware LLMs is not uniform across datasets. While the attacks are particularly effective on Cora, Citeseer, and PubMed, ArXiv exhibits a more nuanced interaction between attack strategy and graph structure, in which poisoning surpasses evasion. This variation underscores that dataset characteristics are integral to evaluating the vulnerabilities of graph-aware LLMs: as these models are deployed across diverse real-world settings, their robustness must be assessed not only with respect to algorithmic design but also with respect to the specific domains and data on which they operate.

Implications for Graph-aware LLM Development The vulnerabilities exposed in this study carry broader implications for the future development and deployment of graph-aware LLMs. Given their accelerating adoption for tasks traditionally handled by GNNs, the tradeoff between performance and vulnerability must become a first-class consideration in the design of these architectures. As graph-aware LLMs continue to evolve, there is a pressing need for more robust encoding techniques and for defenses against adversarial manipulation, particularly feature perturbations. Moreover, characterizing how adversarial attacks transfer from GNNs to graph-aware LLMs opens promising avenues for research into protecting these models against sophisticated adversaries.

Difficulty in Applying Existing GNN defenses Against Adversarial Attacks on Graph-aware LLMs **Defenses on adversarial attacks based on feature perturbation.** Adversarial attacks on GNNs have concentrated almost exclusively on structural perturbations, because feature-based perturbations typically fail to alter the predicted class of the target node (Dai et al. 2018; Wu et al. 2019; Zügner and Günnemann 2019) and are therefore ineffective. As a direct consequence, there exist *no known defenses against feature-based attacks on GNNs*. As demonstrated in the main paper, however, feature perturbations *are* effective within the graph-aware LLM paradigm, and this reversal introduces a genuinely novel challenge: attacks that were inert against GNNs now successfully manipulate model behavior. The shift makes clear the need for new defenses that simultaneously address structural and feature-based perturbations. Text sanitization techniques—for instance, applying optical character recognition (OCR) as a preprocessing step before encoding text for the LLM—offer only partial protection, since OCR is imperfect and misinterprets homographs at a substantially higher rate than unperturbed text (Boucher et al. 2022). This limitation directly motivated our design of an LLM-based self-correcting mechanism for feature perturbations.

Defenses on adversarial attacks based on structural perturbation. Because structure-based adversarial attacks transfer from GNNs to graph-aware LLMs—particularly under

Table 10: Illustration of the imperceptible homoglyph attack. Although the LLM processes the perturbed text as a distinct character sequence, its rendered form is visually near-identical to the original for a human reader, as shown in the image.

Original Feature	What LLM Sees	What a Human Sees
Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.	Title: The mgpprir huristic fr discvring prtln squnc ptttrns Abstract: Svrl cmputr lgrithms fr discvring pttrns in grups f prtln squncs r in us tht r bsd on fitting th prmtrs f sttistical mdl to grup of rltd squncs.	Title: The megaprior heuristic for discovering protein sequence patterns. Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Table 11: Illustration of the imperceptible reordering attack. Although the LLM processes the perturbed text as a distinct character sequence, its rendered form is visually near-identical to the original for a human reader, as shown in the image.

Original Feature	What LLM Sees	What a Human Sees
Title: Applications of machine learning: a medical follow up study. Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.	Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dsceribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiotn of the aplcpiation of three machine larening algorithms-1R, FOIL and InductH to identify risk frcatos that govern the clopssuspension cure rtae has been mdae.	Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

evasion—one might expect GNN defenses to remain effective in this paradigm. This expectation does not hold: unlike traditional GNN settings, *graph-aware LLMs are additionally susceptible to feature perturbations, which renders these defenses ineffective*.

A representative GNN defense makes the adjacency matrix trainable so that edge weights adapt during training to counter adversarial perturbations, assigning lower weights to edges between dissimilar nodes and higher weights to edges between nodes of greater feature similarity. Several **preprocessing-based** defenses build on this principle—for example, removing edges between nodes of low feature similarity (Wu et al. 2019) or introducing randomized edge dropping (Dai et al. 2018). All of them, however, depend critically on clean, unperturbed node features to compute reliable similarity scores. When the features are themselves perturbed, as

in our attacks, these defenses not only fail but can actively worsen the outcome by forging incorrect edges. We demonstrate this effect on a random node in the Cora dataset (Figure 8): whereas the clean feature-similarity graph preserves some original edges alongside a few spurious connections, the graph reconstructed from perturbed features preserves *none* of the original connections, yielding an entirely incorrect reconstruction.

A separate line of work seeks to reduce reliance on the graph structure. For example, (Shanthamallu, Thiagarajan, and Spanias 2021) trains a surrogate predictor that depends solely on node features and uses an uncertainty-matching strategy to extract graph information from the GNN, while Wu et al. (2021) proposes a co-training framework that trains submodels independently on the feature and structural views and distills knowledge by admitting confidently labeled

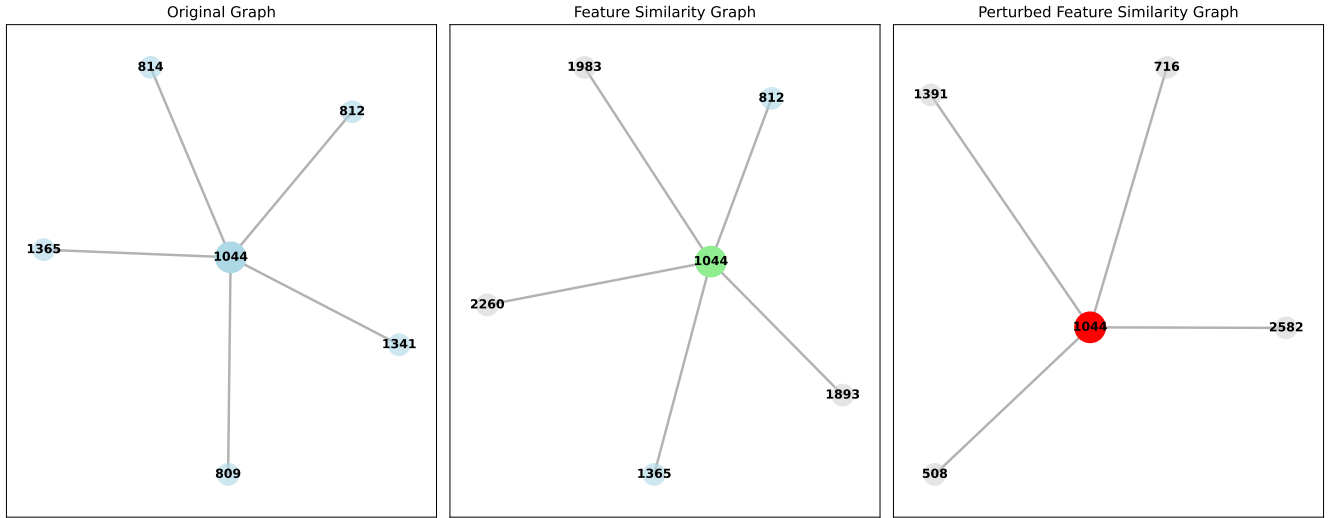


Figure 8: Edges reconstructed for a random node (node 1044). **Original (left):** the ground-truth edges from the original graph. **Feature Similarity (middle):** edges reconstructed from clean node features, with blue nodes marking connections preserved from the original graph. **Perturbed Feature Similarity (right):** edges reconstructed from perturbed node features, which preserve none of the original connections—illustrating why similarity-based GNN defenses fail once features are perturbed.

unlabeled data into the training set. Yet these approaches, too, presuppose access to unperturbed node features—an assumption that fails in our setting, where both structure and features are adversarially perturbed. Consequently, structural defenses developed for GNNs do not transfer directly to the graph-aware LLM paradigm, which demands the novel, paradigm-specific defense we propose.